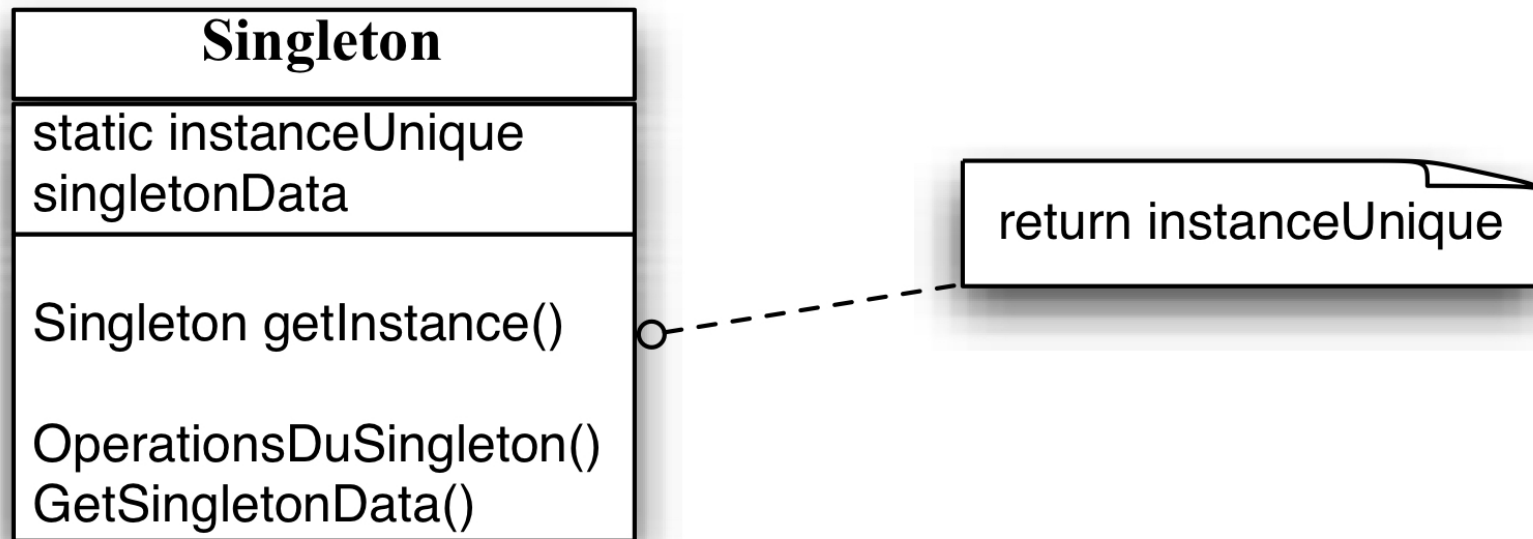


Singleton

Définition

- Un pattern créationnel
- assure qu'il y'a une et une seule instance d'une classe et fournit un point d'accès global vers cette instance
- Exemples d'utilisation:: votre système a besoin d'un seul gestionnaire de fenêtre, un seul spool d'impression, un seul point d'accès vers un moteur de base de données..etc

Diagramme de classe



Création d'une classe Singleton à l'aide d'une méthode statique:

- Pour implémenter le pattern Singleton, il existe différentes approches, mais la majorité possèdent en commun les concepts suivant:
- Le constructeur de la classe singleton doit être déclaré Private, pour interdire son instanciation depuis une autre classe
- Déclarer une variable statique private du type de la classe singleton, qui va-t-être la seule instance de la classe
- Déclarer une méthode statique public, qui va retourner une instance de la classe singleton, cette méthode représente le point d'accès global pour les classes externes afin d'obtenir une instance de la classe singleton

Exemple

```
public class PrintSpooler {  
    private static PrintSpooler spooler;  
    private PrintSpooler() { }  
    public static synchronized PrintSpooler getSpooler() {  
        if (spooler == null) // instance pas encore créé  
            spooler = new PrintSpooler(); //créer une instance  
        return spooler; }  
}  
//le mot clé synchronized est pour ne pas permettre un accès  
//concurrents; i.e deux thread en même temps ne pourront pas  
//accéder à cette méthode
```

Avantage de l'utilisation de la méthode statique

- Pas de gestion d'exceptions
- Impossible de créer directement des instances de la classe car le constructeur private:
 - `PrintSpooler pr3 = new PrintSpooler();`
//génère une erreur de compilation
- Il est facile de modifier la classe pour permettre l'existence de 2 ou 3 instances

Autre méthode de création du Singleton,
en utilisant les exceptions:

- Il est préférable de créer sa propre Exception:
- **public class SingletonException extends RuntimeException {**
- //nouveau type d'exception pour les classes singleton
public SingletonException() { super(); }
public SingletonException(String s)
{ super(s); } }

Singleton avec Exception:

```
public class Spooler {  
    //c'est un prototype de de classe printerSpooler  
    //qui garantit qu'une seule instance peut exister  
  
    static boolean instance_flag = false;  
    //true si une seule instance  
  
    public Spooler() throws SingletonException {  
        if (instance_flag)  
            throw new SingletonException("Seulement une impression  
            permise!");  
        else  
            instance_flag = true; //set flag pour une instance  
            System.out.println( "impression ouverte"); } }
```

Exemple d'utilisation de la classe Spooler

```
public class singleSpooler{  
  
  static public void main(String argv[])  {  
    Spooler pr1, pr2;  
  
    //ouvre une seule impression—doit être réussite  
    System.out.println("Ouverture d'une seule Impression");  
    try {   pr1 = new Spooler();}  
    catch (SingletonException e) { System.out.println(e.getMessage()); }  
  
    //essai d'ouvrir une autre impression—doit echouer  
    System.out.println("Opening two spoolers");  
    try { pr2 = new Spooler();      }  
    catch (SingletonException e) {System.out.println(e.getMessage()); } } }
```